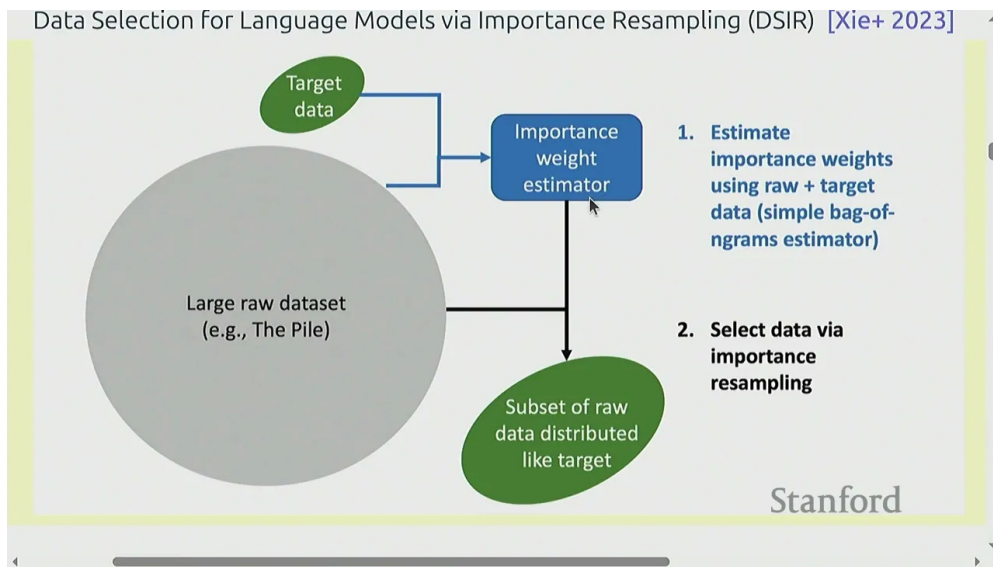


课程笔记

CS336 Lecture 14: 数据过滤与去重

基于授课内容整理

2025 年春季



视频作者/频道: Stanford CS336

发布日期: 2025 年春季

视频时长: 约 30 分钟

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：数据不是从天上掉下来的	3
2 过滤：粗糙但高吞吐的打分器	3
2.1 问题的本质	3
2.2 通用过滤模板	4
2.3 分数、阈值和重采样	4
3 KenLM：用语言概率筛数据	5
3.1 n-gram 语言模型	5
3.2 Perplexity 作为打分	5
3.3 Kneser-Ney 的具体形式	6
3.4 句子级和文档级	6
3.5 阈值怎么定	6
3.6 CCNet 的经验	7
3.7 排序筛选的实际含义	7
4 fastText：用轻量分类器做过滤	7
4.1 bag-of-words 的参数问题	7
4.2 词嵌入平均	8
4.3 ngram 和 hashing trick	8
4.4 在过滤里的角色	9
4.5 训练目标与概率校准	9
4.6 一个过滤器的工作方式	9
4.7 为什么小模型就够	10
4.8 误差结构怎么读	10
5 DSIR：importance resampling	11
5.1 重要性采样复习	11
5.2 为什么重要性采样是对的	11
5.3 从分布到文档	11
5.4 为什么它有时优于 heuristic classifier	12
5.5 实现时要小心什么	12
5.6 和 heuristic filter 的取舍	13
6 三类应用：语言、质量与毒性	13
6.1 语言识别	13
6.2 质量过滤	13
6.3 毒性过滤	14
7 去重：从精确重复到近重复	15
7.1 为什么去重要单独讲	15
7.2 精确重复	15
7.3 Bloom filter	16

7.4	Jaccard 和 MinHash	16
7.5	LSH: 把概率差异放大成阈值效应	17
7.6	shingle 选择与归一化	18
7.7	LSH 参数怎么选	19
7.8	去重与过滤的顺序	19
8	把这一切统一起来	19
8.1	一个稳定的工程模式	19
9	实战选型: 如果你来搭管线	20
9.1	一个示意性的生产管线	21
9.2	最常见的失败模式	21
9.3	如果把整条管线串起来	22
10	本章小结	22

1 导言：数据不是从天上掉下来的

上一讲已经把数据来源、抓取和清洗的大图景铺开了。这一讲要回答的不是“数据从哪里来”，而是更具体的三件事：怎么从海量原始数据里挑出像目标数据的子集，怎么把这套机制复用到不同过滤任务里，以及怎么在近重复规模下保持线性级别的效率。

本讲的判断标准

1. 结果要足够像目标数据，但不能只是复制目标数据。
2. 方法要足够快，能在网页级别数据上跑完。
3. 方法要足够粗糙，粗糙到可以规模化，但又不能粗糙到失去控制。

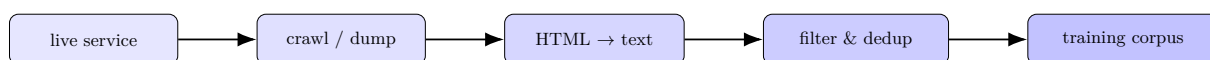


图 1: 从活跃服务到训练语料¹

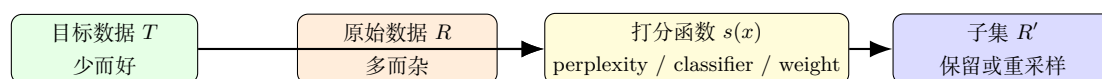


图 2: 过滤问题的抽象：从少量目标数据和大量原始数据里学出一个可打分的选择器

为什么这个抽象很通用

KenLM、fastText、DSIR、语言识别、质量过滤、毒性过滤、近重复召回，本质上都可以塞进同一个框架：先学一个粗糙但快的打分器，再根据分数决定保留哪些样本。

2 过滤：粗糙但高吞吐的打分器

2.1 问题的本质

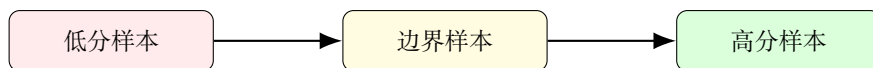
把过滤写成一个公式化问题并不复杂。给定目标数据 T 和原始数据 R ，我们想要一个子集 $R' \subset R$ ，让它在某个意义上更接近 T 。关键不在于“精确拟合”，而在于：

- 你要学会对样本打分；
- 你要把高分样本留住；
- 你要在超大规模数据上完成这个动作。

为什么不能直接上大模型

如果过滤器本身和训练模型差不多重，那么你在筛掉 99% 数据之前，就已经为这 99% 数据付出了接近训练级别的算力。那样做通常不划算。

¹字幕 00:00:05–00:01:25。讲者强调数据来自 live services，之后要经历 crawl/dump、HTML 转文本、过滤与去重。



阈值越高，保留越少
但质量通常越高

图 3: 过滤的核心动作是阈值化或重采样：把连续分数转成是否保留的决定

2.2 通用过滤模板

```

1 model = fit(target_data)
2 scores = [model.score(x) for x in raw_data]
3 keep = [x for x, s in zip(raw_data, scores) if s >= threshold]
  
```

Listing 1: 一个极简的过滤模板

这个模板几乎就是本讲所有方法的共同骨架。

差异只在于打分函数 `score(x)` 是什么，以及 `fit(target_data)` 用什么模型来实现。

2.3 分数、阈值和重采样

同一个分数，在工程里可以有三种常见用法。

用法	动作	适合场景	代价
threshold	高于阈值就保留	噪声过滤、毒性过滤	容易受校准影响
ranking	取 top- k 或 top- $p\%$	CCNet 风格排序、预算固定	结果依赖样本总量
resampling	按权重重采样	DSIR、分布匹配	统计波动更大

表 1: 同一个分数函数，可以驱动三种不同的数据选择动作

为什么要区分这三种动作

阈值适合“要还是不要”的粗过滤；排序适合固定预算下的高质量截断；重采样适合想让样本整体分布更像目标分布的场景。把它们混在一起，往往会导致目标不清晰。

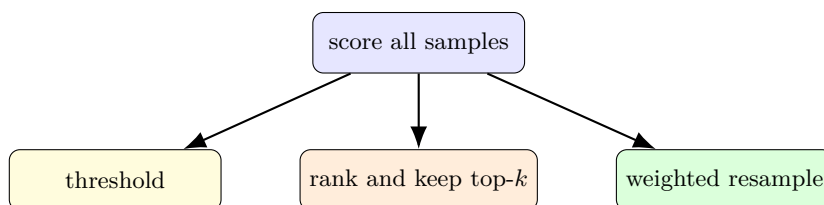


图 4: 同一组分数可以派生出三种选择策略：阈值、排序和重采样

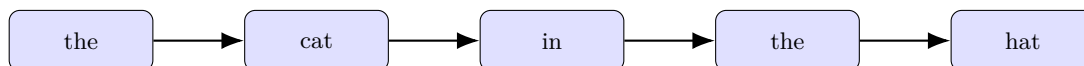
3 KenLM: 用语言概率筛数据

3.1 n-gram 语言模型

KenLM 在数据过滤里的作用是一个很典型的例子：它并不追求最聪明，但它足够快。对一个三元模型，最大似然估计可以写成

$$p(w_i | w_{i-2}, w_{i-1}) = \frac{\text{count}(w_{i-2}, w_{i-1}, w_i)}{\text{count}(w_{i-2}, w_{i-1})}.$$

这就是“计数再归一化”。思想简单，工程上也容易并行化。



条件概率由前两个词决定
n-gram 就是在数这种局部上下文

图 5: n-gram 的直觉：用有限窗口的上下文预测下一个词

Kneser-Ney 的作用

当高阶上下文稀疏到不可信时，就回退到更短的上下文。它不是“凭空补概率”，而是在统计证据不足时，把判断交给更稳的低阶模型。

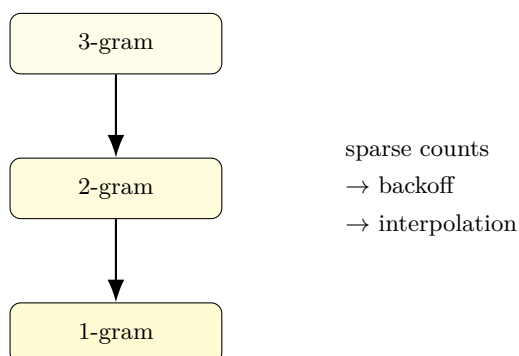


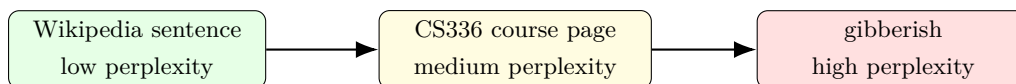
图 6: Kneser-Ney 的核心直觉：高阶不可靠就回退，短上下文提供更稳定的统计支撑

3.2 Perplexity 作为打分

对于一段文本 $x = w_1, \dots, w_n$ ，perplexity 的形式常写成

$$\text{ppl}(x) = \exp\left(-\frac{1}{n} \sum_{i=1}^n \log p(w_i | w_{<i})\right).$$

它越低，说明文本越像训练出来的语言模型认为“自然”的文本。KenLM 用它过滤网页时，目标不是判断这段话是否“有价值”，而是先剔除那些明显不像自然语言的垃圾片段。



perplexity 上升意味着“更不像训练语料”

图 7: perplexity 的过滤意义：不是审美判断，而是“像不像自然文本”的粗粒度度量

KenLM 的边界

它擅长抓“明显不像人话”的内容，但不擅长判断逻辑、事实、长程一致性和内容价值。它更像一个廉价的门卫，而不是一个老师。

3.3 Kneser-Ney 的具体形式

如果把回退写得更具体一点，常见形式是

$$p_{\text{KN}}(w_i | h) = \frac{\max(c(h, w_i) - d, 0)}{c(h)} + \lambda(h) p_{\text{KN}}(w_i | h'),$$

其中 h 是当前上下文， h' 是更短的回退上下文， d 是折扣项。前一项保留显式计数，后一项把剩余概率质量交给低阶模型。

为什么折扣项重要

如果不折扣，高阶计数会把概率全吃光；如果折扣过大，高阶上下文又失去区分力。Kneser-Ney 的价值就在于：它用一个明确的概率守恒机制，把“高阶证据”和“低阶稳健性”拼起来。

3.4 句子级和文档级

实践里，KenLM 可以给句子打分，也可以给整篇文档打分。前者更容易发现局部乱码和拼接噪声，后者更适合做网页级排序。很多系统会把两者组合起来：先删掉单句极端异常的段落，再在文档层面做聚合。

粒度	优点	缺点	典型用法
sentence	能抓局部异常	容易受短句波动影响	先做粗清洗
paragraph	能保留局部上下文	需要更长文本	CCNet 风格排序
document	更适合网页级管线	可能掩盖局部噪声	终局保留决策

表 2: KenLM 不同打分粒度的取舍

3.5 阈值怎么定

KenLM 的阈值通常不靠“感觉”拍脑袋，而是看一小批人工检查样本上的分布差异。最常见的做法是先看目标数据和噪声数据的 score 分布，再选择能把两者拉开的 operating point。

阈值的一个常见误区

如果你只看一个绝对阈值，而不看不同域的分数的分布，那么阈值会严重迁移失配。网页、书籍、代码、聊天记录的 perplexity 尺度都可能不一样。

3.6 CCNet 的经验

课程里提到的 CCNet 做法很朴素：以段落为单位，按 perplexity 排序，保留前 1/3。这个规则有点粗暴，但它能以极低成本把大批低质量网页剔除。后来 LLaMA 的数据管线也受到了这种思路的影响。

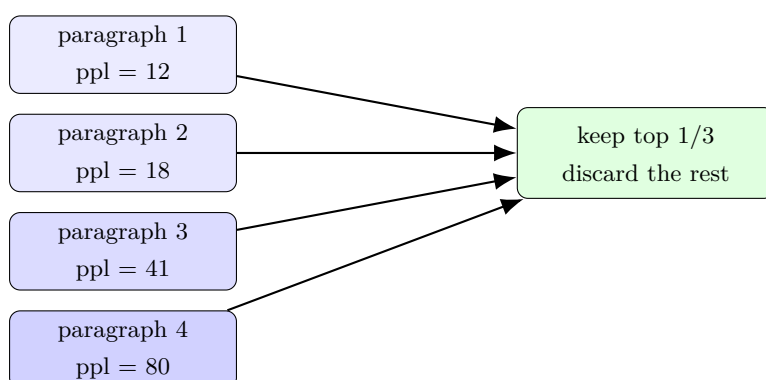


图 8: CCNet 风格的排序筛选：把段落按 perplexity 排序，然后保留最像自然文本的一部分

3.7 排序筛选的实际含义

排序和阈值的差别不只是形式。阈值问的是“这条样本够不够好”，排序问的是“在预算有限时，哪一批最好”。后者对大规模语料更常见，因为你常常已经知道总预算，真正的问题是如何在预算内把质量最大化。

排序筛选什么时候更稳

当 score 的绝对值校准不稳定，但相对次序还算可靠时，排序通常比硬阈值更稳。CCNet 一类方法之所以常见，就是因为它们更多依赖相对顺序，而不是绝对概率值。

4 fastText：用轻量分类器做过滤

4.1 bag-of-words 的参数问题

如果把文本直接做 bag-of-words 分类，参数量会非常大。假设词表大小为 V ，类别数为 K ，那么一个朴素线性分类器的权重矩阵就可能接近 $V \times K$ 。对大词表来说，这很快就变成稀疏且昂贵的系统。

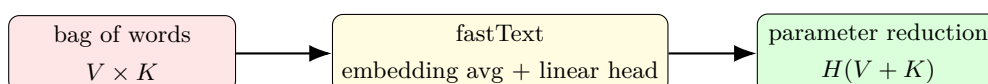


图 9: fastText 的核心：把高维稀疏问题压缩成低维表示，再用线性头做分类

fastText 为什么适合过滤

- 推理快，适合扫描海量网页。
- 训练快，适合快速迭代阈值。
- 线性结构足够稳，适合二分类过滤。

4.2 词嵌入平均

fastText 的关键就是把词表示成低维 embedding，然后把一个句子的 embedding 做平均，最后接线性分类器。表面看起来简单，实则把“词表巨大”这个问题变成了“维度可控”的问题。

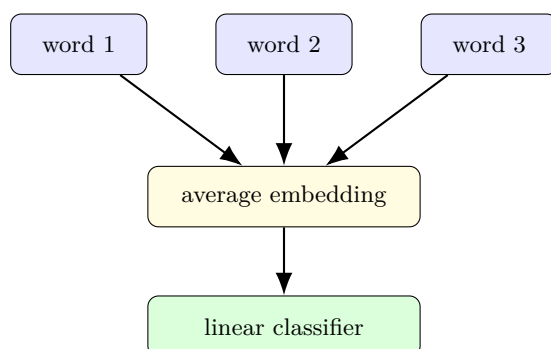


图 10: fastText 的前向路径：词嵌入平均后直接线性分类，没有复杂非线性

4.3 ngram 和 hashing trick

为了捕捉局部短语信息，fastText 还会加入 bigram、trigram 等 n-gram 特征。问题是 n-gram 维度会爆炸，所以要用 hashing trick 把无限词组压进有限桶里。

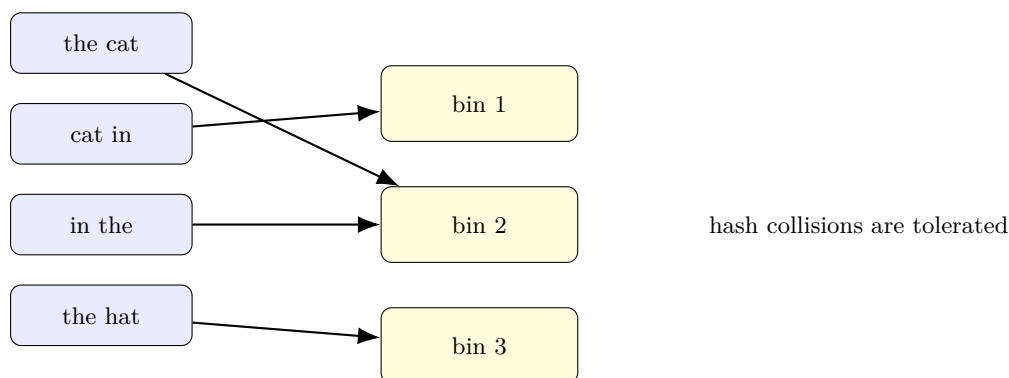


图 11: hashing trick 的作用：把不受控的 n-gram 空间压缩到固定桶数里

fastText 的现实限制

它本质上仍是一个线性模型。它能快地做“像不像目标类”的判断，但不会自动理解篇章结构、逻辑严密性或事实正确性。

4.4 在过滤里的角色

如果 KenLM 更像“语言门卫”，fastText 就更像“轻量分类器门卫”。前者用语言模型分数过滤，后者直接预测文档是否属于目标类。比如语言识别、数学文本过滤、毒性识别，都可以用这个思路。

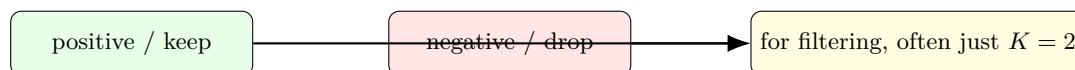


图 12: 在过滤任务中，fastText 经常退化成二分类器：保留还是丢弃

4.5 训练目标与概率校准

fastText 通常优化的是交叉熵：

$$\mathcal{L} = - \sum_{(x,y)} \log p_{\theta}(y | x).$$

如果类别不平衡，单纯看准确率没有意义。过滤场景更关心的是 precision、recall 和阈值后剩余样本的质量。换句话说，你不是在做考试排名，而是在决定哪些数据值得保留。

指标	说明	过滤里的作用	注意点
accuracy	整体预测对不对	不够敏感	类别不平衡时失真
precision	保留下来的样本有多干净	很重要	常和阈值联动
recall	目标样本保住了多少	防止过度过滤	不能只追高 precision
calibration	概率是否可信	用于阈值选择	域外常会漂移

表 3: fastText 过滤任务里真正有用的指标，不是单纯 accuracy

4.6 一个过滤器的工作方式

你可以把 fastText 当成一个“快速排序器”。它不会替你理解文本，而是把一堆样本按一个近似相关的分数排好，然后让你在高分区间做保留决策。

1. 先准备少量标注数据，定义什么叫正样本、负样本。
2. 训练一个轻量线性分类器，让它学会粗略区分两类文本。
3. 把模型跑在海量网页上，得到每个样本的置信度。
4. 在验证集上挑阈值，决定保留多少数据、牺牲多少召回。

为什么这个流程很稳

它把“建模”拆成了两个层次：一个是轻量预测器，另一个是阈值控制器。预测器解决次序问题，阈值控制器解决预算问题。两者分开以后，调参会比把所有东西揉成一个大模型更清楚。

4.7 为什么小模型就够

fastText 在这里不是为了“理解文本”，而是为了在海量样本上快速给出一个稳定的秩序。只要排序足够好，很多过滤决策就已经足够用了。再往上堆复杂模型，收益往往不如增加标注、调整阈值和清理输入稳定。

模型复杂度	好处	坏处	在过滤里的建议
线性/fastText	便宜、快、稳定	表达能力有限	优先考虑
小型 Transformer	更强上下文建模	推理成本高	只在高价值子任务用
大模型打分器	语义强、泛化强	很贵、很慢	通常不划算

表 4: 过滤任务中，模型越强不一定越合适

fastText 常见的工程优势

- 推理快，适合扫描海量网页。
- 训练快，适合快速迭代阈值。
- 线性结构足够稳，适合二分类过滤。
- 可以用少量标注快速建立基线。

4.8 误差结构怎么读

在过滤场景里，fastText 的假阳性和假阴性都有明确代价。假阳性意味着把坏样本留进来了，假阴性意味着把好样本扔掉了。前者污染训练集，后者降低数据效率。实际系统通常会在这两者之间做偏向性的取舍。

错误类型	含义	后果	常见应对
false positive	坏数据被保留	污染训练集	提高阈值
false negative	好数据被丢掉	数据利用率下降	降低阈值 / 二阶段过滤
calibration drift	概率不再可信	阈值失效	重新校准

表 5: fastText 过滤里的三类典型错误与应对

过滤任务里的“好”不是绝对的

一个分类器如果在测试集上看起来很准，但把你真正想保留的长尾样本大量过滤掉，那它在数据工程里依然可能是坏的。过滤的目标不是分类正确率，而是训练集收益。

5 DSIR: importance resampling

5.1 重要性采样复习

DSIR 的关键直觉来自重要性采样。假设你想从目标分布 p 采样，但你只能从 proposal 分布 q 采样，那么你先从 q 抽样，再用权重

$$w(x) \propto \frac{p(x)}{q(x)}$$

重新加权，最后再按权重重采样。这样得到的样本集合会更接近 p 。

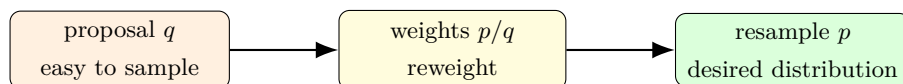


图 13: importance resampling 的基本三步：先采样，再重权重，最后重采样

DSIR 的价值

和只做二分类相比，显式估计 p/q 更接近“我到底想要什么数据”的原始目标。它会更自然地兼顾多样性与覆盖性，而不仅仅是挑出最像的那批。

5.2 为什么重要性采样是对的

如果你关心的不是单个样本，而是某个函数 $f(x)$ 在目标分布下的期望，那么重要性采样给出的是

$$\mathbb{E}_p[f(x)] = \frac{\mathbb{E}_q[w(x)f(x)]}{\mathbb{E}_q[w(x)]}.$$

这条式子说明：只要权重 $w(x)$ 估得够好，你就可以从 proposal 分布的样本上，近似恢复目标分布的统计量。

这和过滤的关系

过滤不是必须严格“采样到目标分布”，但如果你能让保留的数据在统计上更接近目标分布，那么后续训练通常会更稳。DSIR 的价值就在于它把“像不像”升级成了“分布像不像”。

5.3 从分布到文档

真正的数据选择不是在理想分布上抽象地采样，而是在小目标数据集 D_p 和大原始数据集 D_q 上落地。DSIR 的难点也在这里： D_p 太小，没法直接训练复杂分布模型。所以课程里给出的办法是把文本先做 hash n-gram，再估计低维分布。

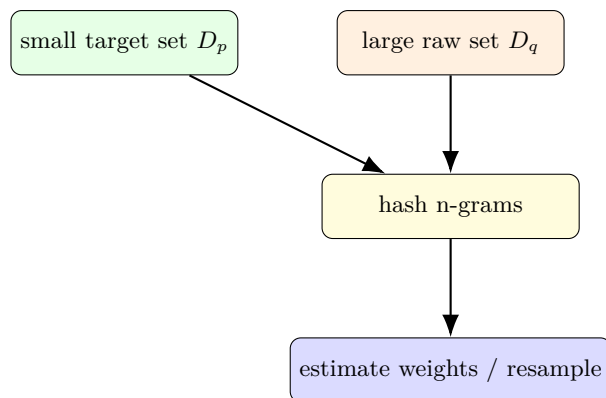


图 14: DSIR 的工程落点：把难以建模的文本分布，压缩到 hash n-gram 空间里估权重

DSIR 不是银弹

它仍然依赖简化表示。如果 hash n-gram 太粗，权重就会失真；如果特征太细，目标数据又会不够支撑估计。它只是把“更原则化”的问题变成一个更可控的近似。

5.4 为什么它有时优于 heuristic classifier

fastText 的做法很像“判定这个样本像不像目标类”，而 DSIR 更像“让样本集整体更接近目标分布”。后者在多样性和覆盖性上通常更自然，也更适合用于构造更均衡的数据子集。

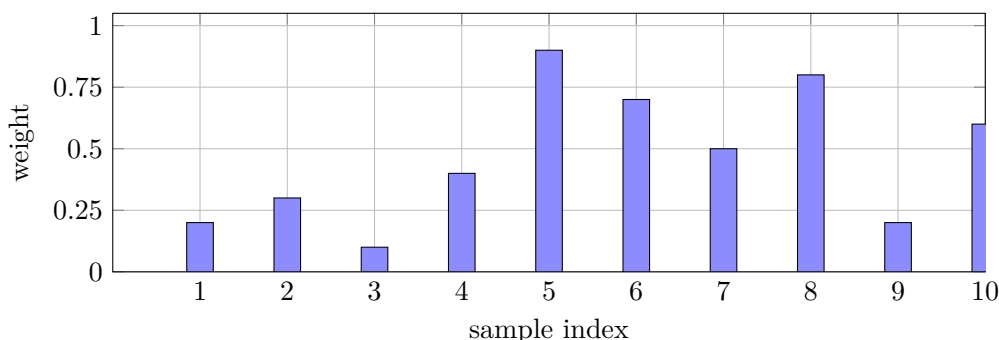


图 15: importance weight 的直观含义：proposal 里抽到的样本并不均匀，需要按权重修正

5.5 实现时要小心什么

实做 DSIR 时，最容易出问题的是权重极端不均匀。少数样本权重特别大，会导致重采样几乎只剩它们，结果失去多样性。常见做法包括权重截断、归一化、平滑和分桶。

问题	现象	常见处理
权重过尖	少数样本统治结果	clipping / smoothing
特征过稀	估计方差很大	降维 / hash / 合并桶
目标集太小	权重不稳定	先做粗过滤再 DSIR

表 6: DSIR 的几个典型失败模式和应对方式

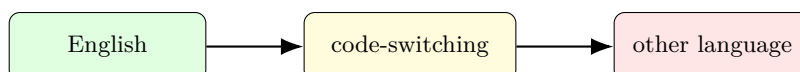
5.6 和 heuristic filter 的取舍

如果你只想快速删掉一大批明显不相关的网页，heuristic classifier 往往够了。但如果你要认真控制保留下来的数据分布，比如想让某个主题、体裁或领域的比例更合理，DSIR 通常更符合目标。

6 三类应用：语言、质量与毒性

6.1 语言识别

语言识别的目标非常实用：先把非目标语言筛掉，再把算力留给真正关心的语言。对很多网页语料工程来说，这一步比“更聪明的模型”更重要，因为语料分散在多种语言、方言和代码混排里。



语言识别在边界处最难

图 16: 语言识别的难点不是单语句子的“标准英语”，而是边界处的混合文本和低资源变体

语言识别的现实难题

- 短文本缺少上下文。
- 低资源语言样本少。
- code-switching 的语言边界并不稳定。
- 相近语言和方言容易互相混淆。

课程里提到 fastText 语言识别支持 176 种语言，Dolma 则用 $p(\text{English})$ 作为一个简单门槛。这个门槛不一定很“科学”，但它够快，能把大批明显不是英语的页面先去掉。

6.2 质量过滤

质量过滤并不等于“语法正确”，它更像是在判断一页文本是否值得进入训练集。常见被剔除的对象包括模板页、广告页、低信息密度页、重复页、乱码页和明显的拼接页。

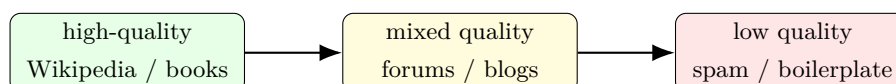


图 17: 质量过滤是沿着“信息密度”而不是“语法正确性”做粗粒度排序

质量过滤的本质

它不是定义“完美文档”，而是决定“哪些数据更值得被训练”。只要能持续过滤掉最差的那一截，收益通常就会很明显。

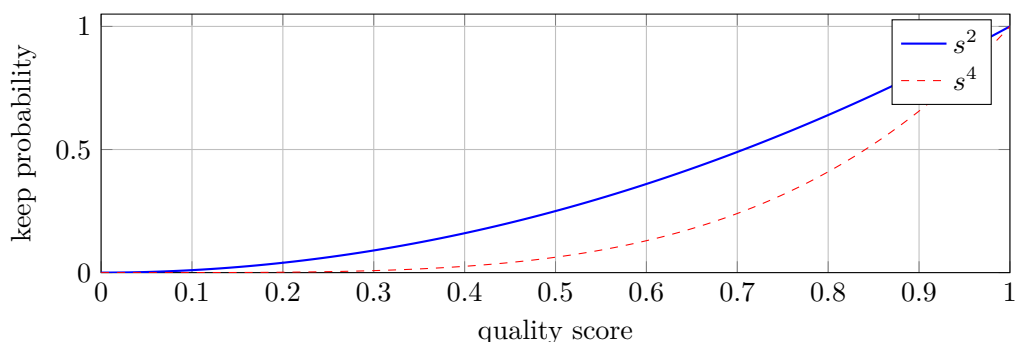


图 18: 像 GPT-3 这样的随机保留策略，可以用更陡的曲线把高分样本保留下来、低分样本快速压下去

GPT-3、LLaMA/RedPajama、phi-1 的差异在于正负样本选择不同，但骨架是一样的：先训练一个轻量分类器或打分器，再按分数筛选。phi-1 的有趣之处在于，它把“教育价值”作为过滤准则，用教材和高质量合成样本去筛大规模代码数据。

6.3 毒性过滤

毒性过滤是更直接的治理场景。Dolma 使用 Jigsaw Toxic Comments 数据训练轻量分类器，把评论分成 toxic、severe_toxic、obscene、threat、insult、identity_hate 等类，再按需求过滤或标记。

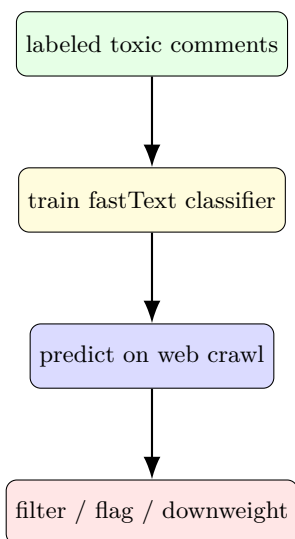


图 19: 毒性过滤的典型管线：用带标签数据训练轻量分类器，再把它应用到网页级数据上

毒性过滤的坑

误杀经常出现在短文本、引号引用、讽刺、代码混排和方言里。一个看似“更严格”的分类器，可能只是把更多正常内容误删掉。

7 去重：从精确重复到近重复

7.1 为什么去重要单独讲

过滤解决的是“留下什么”，去重解决的是“别重复留下什么”。它特别重要，因为重复数据会导致训练效率下降、记忆化上升，甚至引入版权和隐私问题。课程里把去重分成精确重复和近重复两类，这个划分非常关键。

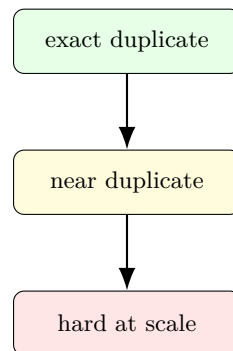


图 20: 去重从精确重复走向近重复后，问题会迅速从“简单 hash”升级为“可扩展的相似度检索”

7.2 精确重复

精确重复最简单：按 hash 分组，每组留一个。MapReduce 风格天然适合这个任务，因为你可以先局部 hash，再全局 merge。C4 常见的做法是以 3-sentence span 为单位做精确去重，简单但有效。

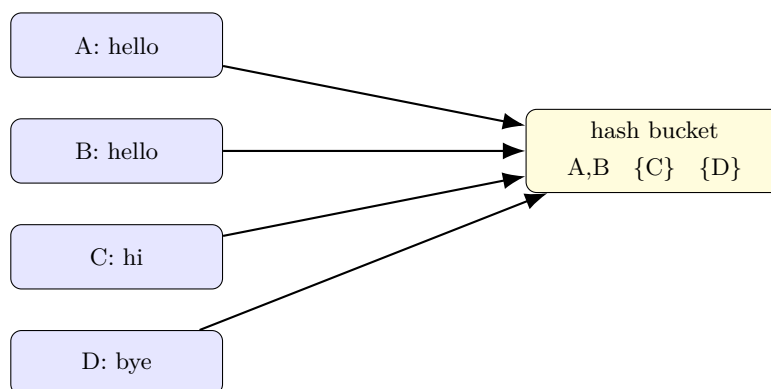


图 21: 精确去重的最小原型：把 hash 一样的样本分到同一桶里，每桶保留一个

精确去重的优点

- 语义清楚。
- 高精度。
- 非常容易并行化。

精确去重的缺点

它抓不住只差几个 token 的近重复，也可能把文档中间的一段切掉，导致剩余上下文不连贯。

7.3 Bloom filter

Bloom filter 是一个典型的工程妥协：它用少量误报换大量空间节省。它适合快速判断“这个 item 有没有出现过”，但不适合精确删除。

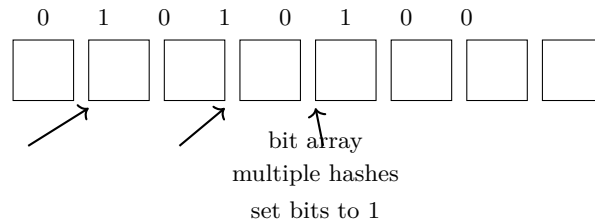


图 22: Bloom filter 的基本结构：若干哈希把元素映射到 bit array 上的多个位置

如果用 m 个桶、 k 个哈希函数、插入 n 个元素，假阳性率近似为

$$f \approx (1 - e^{-kn/m})^k.$$

在固定 m/n 下，最优的 k 大约是

$$k \approx \frac{m}{n} \ln 2.$$

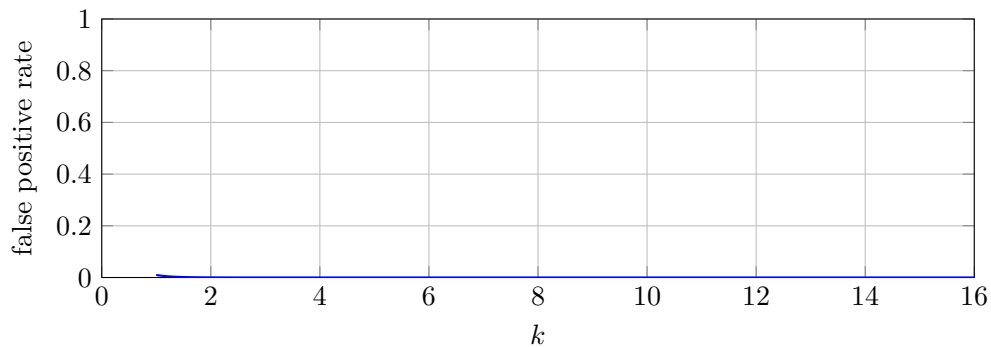


图 23: Bloom filter 的参数权衡：哈希函数越多，误报率先降后升，存在一个最优点

Bloom filter 的本质

它不是要“绝对正确”，而是用少量误报换极低内存占用。对网页级去重来说，这通常是合理的。

7.4 Jaccard 和 MinHash

近重复检测通常要先定义相似度。最常见的是 Jaccard：

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

如果两个文档的 Jaccard 足够高，就可以把它们看成近重复候选。但难点在于：怎么不做全量两两比较。

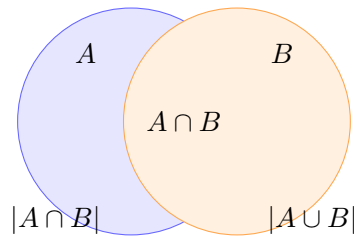
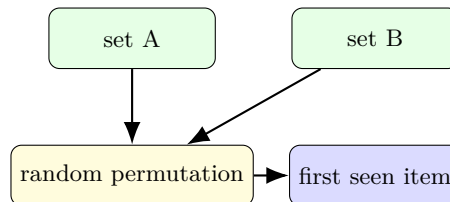


图 24: Jaccard 的直觉：看两个集合的重叠程度，而不是单独看每个集合有多大

MinHash 的关键性质是：对于随机哈希函数 h ,

$$\Pr[h(A) = h(B)] = \text{Jaccard}(A, B).$$

这意味着你不用显式算所有集合相似度，只要看“是否碰撞”就能估计近重复关系。



相似集合更容易在同一位置“先出现”

图 25: MinHash 的直觉：把集合做随机排列，再看最先出现的元素是否一致

MinHash 的作用

它把相似度问题变成了“碰撞概率”问题。概率一旦可估计，就能进一步用来做大规模索引和候选召回。

7.5 LSH：把概率差异放大成阈值效应

LSH 的核心是 banding。把 n 个 hash 分成 b 个 band，每个 band 有 r 个 hash，满足 $n = br$ 。若相似度为 s ，那么某个 band 匹配的概率是 s^r ，至少有一个 band 匹配的概率是

$$1 - (1 - s^r)^b.$$

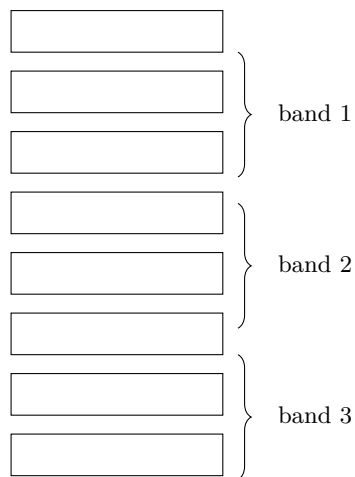


图 26: LSH 的 banding 思路：把多个 hash 分块，任何一块完全相同都能触发候选对

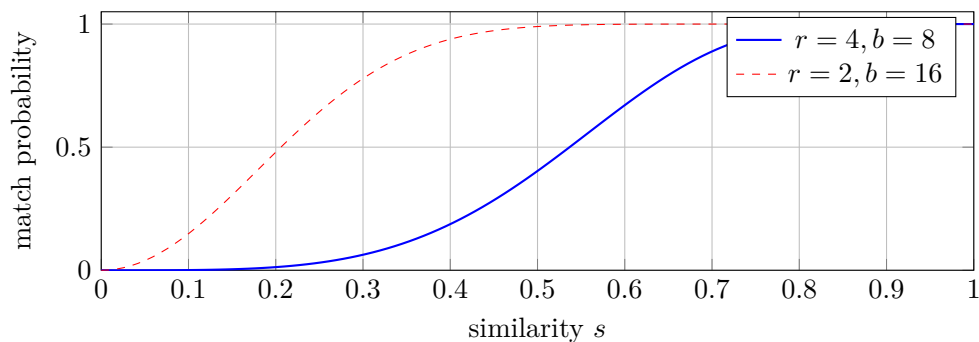


图 27: LSH 的阈值效应：不同 banding 参数会让曲线在不同相似度位置陡然上升

LSH 的工程意义

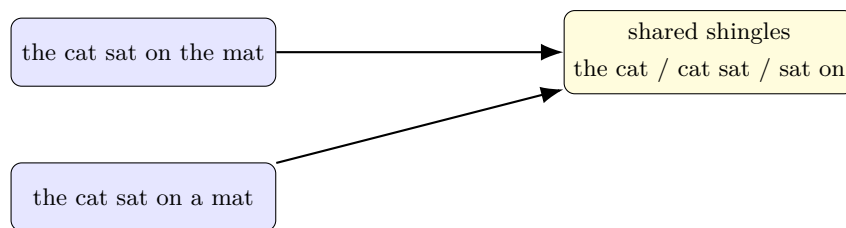
- 先哈希分桶，再做局部比较。
- 把连续相似度问题变成碰撞问题。
- 对大规模近重复召回非常合适。

7.6 shingle 选择与归一化

MinHash 和 LSH 并不是直接对原始字符串做，而通常是先把文本切成 word shingles 或 character shingles，再做集合化表示。选择多少 token 一组，不是纯数学问题，而是对语言形态、编辑差异和噪声类型的折中。

shingle 类型	优点	缺点
word shingles	更符合语义块	对标点和分词敏感
char shingles	对拼写和局部改写更稳	可能太细、噪声更多
sentence spans	适合整段去重	切分误差会放大

表 7: 近重复检测里最先要想清楚的，不是算法，而是 shingle 怎么定义



轻微改写后仍保留大量共享 shingle

图 28: 近重复检测的直觉例子: 只改一个词, 仍然会留下足够多的共享片段

7.7 LSH 参数怎么选

LSH 的参数本质上在调召回和精度的分界线。 r 越大, 单个 band 越严格; b 越大, 触发候选的机会越多。它们共同决定曲线在什么相似度附近拐起来。

参数变化	效果	代价	适合场景
增大 r	更严格, 更少误报	容易漏掉近重复	候选太多时
增大 b	更容易触发候选	召回更高但更吵	不想漏掉时
减小 r	更宽松, 更多召回	候选数上升	高价值近重复

表 8: LSH 选参时, r 和 b 共同决定候选召回曲线

实际选参思路

如果你更怕漏掉近重复, 就加大 b 或减小 r ; 如果你更怕候选太多, 就提高 r 。这个选择通常和数据规模、可接受的候选对数量直接相关。

7.8 去重与过滤的顺序

过滤和去重经常串联, 但顺序并不总是固定。一般来说, 先做语言识别和明显质量过滤, 再做去重, 会更省算力; 但如果你有很强的重复污染, 先做粗去重也可能更稳。真正合理的答案不是“一刀切”, 而是看你的数据源结构和预算。

一个常见误区

不要把“去重”理解成简单删掉重复句子。对于网页和文档, 重复结构常常是局部的、分散的、跨段落的。真正有效的近重复检测必须配合 shingle、banding 和候选召回一起看。

8 把这一切统一起来

8.1 一个稳定的工程模式

KenLM、fastText、DSIR、Bloom filter、MinHash、LSH, 看起来像不同主题, 实际上都在复用同一类工程模板: 先用便宜的统计量压缩问题, 再把大规模搜索变成分桶、排序、阈值判断或者重采样。

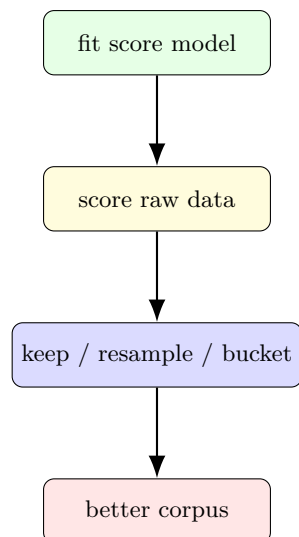


图 29: 统一框架：先训练一个粗糙但快的打分器，再把它应用到大规模原始数据上

方法	score	action	典型用途
KenLM	$p_T(x)$ / perplexity	threshold / sort	language / quality filtering
fastText	$p(T x)$	threshold / classify	language / toxicity / quality
DSIR	$p_T(x)/p_R(x)$	weighted resampling	more principled data selection
Bloom filter	membership bits	approximate reject	exact dedup
MinHash LSH	+ collision probability	candidate recall	near dedup

表 9: 同一套机制在不同任务里的落点

最重要的现实约束

这些方法不是在追求“数学上最优”，而是在追求“算得起、筛得动、错得起”。大规模数据处理的核心，就是在 compute、memory、误杀率之间做足够清楚的取舍。

9 实战选型：如果你来搭管线

把前面的内容收束成一个更实用的问题：面对一批新语料，你到底该先上什么、后上什么、谁负责做主决策？下面这张表是一个很粗但好用的起点。

目标	优先方法	为什么	常见补充
语言识别	fastText / 轻量分类器	直接预测类别，速度快	KenLM 做二次筛
低质量网页剔除	KenLM / sorting	对自然语言程度敏感	规则过滤 + 黑名单
毒性过滤	fastText / classifier	有标签数据时最直接	人工审查高风险样本
分布匹配	DSIR	更接近目标分布	先粗过滤再重采样
精确重复	hash / Bloom filter	便宜、可并行	按 span / block 做
近重复	MinHash + LSH	可扩展到海量数据	结合归一化与切块

表 10: 如果要快速搭一条数据管线，可以先从这个选型表开始

9.1 一个示意性的生产管线

下面是一个示意性的数值例子，不是课程里的硬统计，而是为了说明各个模块如何把规模一层层压下去。假设你从 1 亿条网页文本开始：

阶段	剩余量	主要作用	为什么有用
raw crawl	100M	原始网页抓取	噪声最大
language ID	45M	去掉非目标语言	先砍掉明显无关数据
quality filter	18M	去掉低信息密度页	保住真正能学的文本
toxicity filter	15M	去掉风险内容	降低治理成本
exact dedup	12M	去掉完全重复内容	降低记忆化
near dedup	9M	去掉近重复模板页	降低结构性冗余

表 11: 一个示意性的多阶段清洗管线：每一步都在把“大而脏”压成“小而干净”

为什么这种分层通常更合理

因为每一级做的事情都不同。语言识别要的是类别；质量过滤要的是信息密度；毒性过滤要的是风险；去重要的是冗余。把不同目标分开，既更清楚，也更容易调参。

9.2 最常见的失败模式

失败模式	现象	补救方法
过度过滤	数据变少但质量并没明显变好	降低阈值，检查误杀样本
校准漂移	训练时好用，换域后失效	重新校准或改用排序
重复污染	训练损失很快下降但泛化差	加强 exact / near dedup
候选爆炸	LSH 候选对太多	提高 r 或先做粗过滤

表 12: 数据管线里最常见的几类失败模式

不要把所有事情交给一个模型

一条成熟的数据管线通常是多级的。每一级都只解决一个问题，而且只做它最擅长、最便宜的那部分。把语言识别、质量判断、毒性、去重全压到一个模型里，通常只会得到一个又贵又不稳的系统。

9.3 如果把整条管线串起来

真正的生产系统通常会把这些模块串在一起：先做语言识别，再做质量过滤，再做毒性过滤，再做精确和近重复去重。每一步都不需要完美，但每一步都得足够便宜。

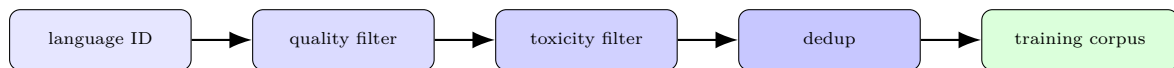


图 30: 端到端数据清洗管线

10 本章小结

这一讲的结论可以压成一句话：**数据处理本身就是建模问题，只不过这里的模型要优先考虑吞吐、空间和误差边界，而不是单纯追求精度。**

三条最终 takeaway

1. KenLM、fastText、DSIR 共享同一类“先打分，再筛选”的过滤范式。
2. 语言识别、质量过滤、毒性过滤，本质上都是为训练集挑选更合适的样本。
3. 精确去重、Bloom filter、MinHash 和 LSH 共同构成了大规模语料去重的工具箱。

真正值得记住的不是某个实现

真正值得记住的是这条线：**先用粗糙模型估计样本值得不值得留下，再用可扩展的近似算法把这个判断做成百万、亿级数据都能跑完的系统。**